Summary Report of HEdClass Web App Project

GitLab reop URL: https://gitlab.eeecs.qub.ac.uk/40270840/hedclass.git

Institutional Administrator login details: Username: registry_admin Password: QUB2026

Classification Officer login details: Username: class_officer1 Password: Queens2026

The goal of this project was to design and implement a Web App called HEdClass, which offers a final degree classification platform for higher education institutions. By automating the classification calculation process, the app aims to ease time pressure on administrative staff, reduce instances of inconsistent rule application and human error and improve scalability for growing student cohorts. Considering the importance of consistency and accuracy, I chose to implement my web app using a technology stack that supports this requirement. I chose to develop the data layer using MySQL due to its focus on data integrity and rigid schema compliance to ensure consistent data for use in calculations. Additionally, MySQL handles highly interconnected entities efficiently through joins, which suitably manage the one-to many relationships between a classification officer and their assigned programmes, a programme and its modules, and a module and its students' grades. Client-side management was done through EJS, HTML and CSS, including the use of the Bootstrap CSS framework, chosen due to its responsive design benefits, consistency and numerous utilities for customisation. The use of Node.js as the runtime environment, Express.js as the web framework and JavaScript as the programming language completes the server-side portion of the technology stack. With accuracy, consistency, and security in mind, I set out to create a user-friendly app that satisfies the requirements highlighted in the specification. In this report, I will detail the final implementation, how it meets the Moscow requirements in the specification and explain key development decisions.

I. System Architecture

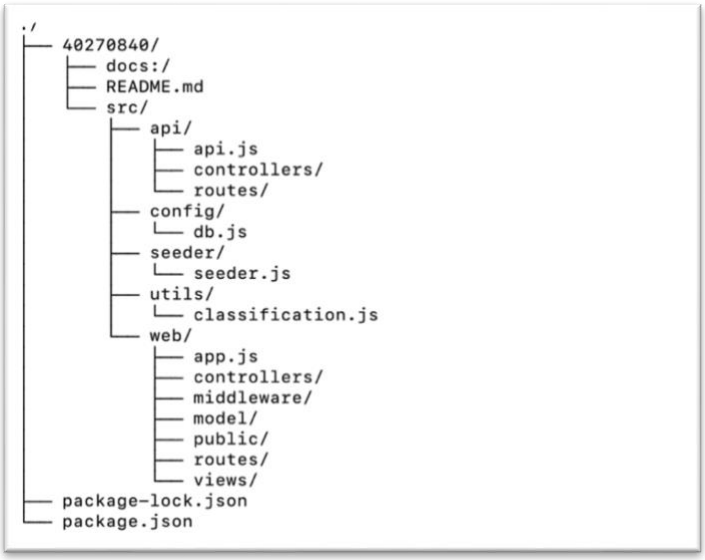


Figure 1: Folder Structure

My Web App satisfies MoSCoW requirement E.1 of the assignment specification by utilising a modular architecture to effectively separate concerns as demonstrated in Figure 1. This was achieved by implementing a standalone REST API with its own controllers and routes to communicate with the database as stipulated in requirement E.2, while the web controllers use fetch requests to retrieve data from the API endpoints. This approach abstracts out the database interaction, decoupling the data handling from the presentation layer and making the data more secure. The seeder folder contains the SQL Drop, Create and Insert statements to fully populate the database with sample data, as well as using bcrypt to hash passwords on insert. Dropping existing tables every time the seeder.js runs ensures that duplicate tables are not accidentally created, supporting data consistency and integrity.

The model folder contains the SQL statements that communicate with the database. Storing all database queries here allows them to be reused throughout the web app and meets the Don't Repeat Yourself programming principle. The API controllers handle the business logic of the web app and sit between the models and the routes, while the web controllers tackle the preparation of this data for use in the user interface. However, I chose to create a separate utils folder to store the classification logic because of its significance to the project and its complexity. Storing it separately allows for easier management and updates.

The API routes folder stores the URL endpoints for returning data, while the web routes are designed for rendering pages for human consumption. These files are then imported into the api.js or app.js file, keeping them clean by delegating the task of path handling. The middleware folder stores the code needed for authentication, including role-based access control and protecting officer assignment data. Finally, the views folder contains the EJS templates, and the public folder contains static files such as images. Utilising the MVC pattern allows the developer to manage their system more effectively by separating concerns and allowing individual updates. It also meets the scalability requirement as outlined in requirement D.2 by making code reusable in additional routes without compromising the integrity of previously established features, which also supports deterministic and consistent rule execution as specified in requirement D.1.

II. Authentication

To ensure base security measures (D.3), a global middleware function was created to be the first point of contact for all users. As stated in requirement A.2, it allows the system to maintain state by ensuring that a valid session object exists and that the user credentials are valid. The login form is then processed by checking the entered username against the users database table and ensuring a password match using bcrypt.compare, which also contributes to requirement D.3. Once user credentials are verified, a session

is created and saved using the express session middleware with an expiration time of 10 minutes to ensure security standards are upheld and data access is protected. I encountered an issue when running the app using nodemon that caused sessions to be wiped upon page updates or reloads. After some research, I discovered that the default storage is not always persistent with nodemon, so to ensure that state was properly maintained, I installed a database-backed session store to ensure that user sessions were persistent.

After session creation, role-based access control is carried out to satisfy requirement A.3. The login post route checks the logged-in user against the user.role_Id column in the table. If it is a 1, the user is an Institutional Administrator and is redirected to the Admin Dashboard. If it is a 2, the user is a Classification Officer and is redirected to their personalised dashboard. Role-based route protection is handled using two dedicated middleware functions, which are then passed into the route parameters as needed to lock the routes. The isAdmin function checks that a session exists for the user and checks that their user.role_is is 1 before allowing them access to the admin-specific routes. The isOfficer function does the same for the classification officer role, and a third function ensures that a classification officer can only access the degree programmes that they are assigned to. If any of these conditions are not met, the user is redirected back to the login page, which ensures protection against bad users as required by D.3. Finally, to close the login/logout cycle, a logout button is available on every page in the web app, so the user can log out at any time. Upon clicking this button, the user's session is destroyed, and they are redirected to the login page, meeting requirement A.1 of the specification.

III. Secondary Actor

The Institutional Administrator's Dashboard was designed with the user's convenience in mind. Buttons provide quick access to repeated actions, directing the user to forms for adding a new officer programme and buttons to view a full list of officers or programmes, fulfilling requirement B.1. The database is queried to fetch arrays of officers and programmes whose lengths are conditionally calculated when the page loads to provide up-to-date statistics regarding assigned officers, unassigned officers, assigned programmes and unassigned programmes to help the administrator manage their time and prioritise tasks. This feature contributes to the fulfilment of a modern user interface as specified in requirement D.1.

The 'add' forms fulfil functional requirements B.1, B.2, B.3 and B.4 by allowing the administrator to create classification officers, assign them to more than one degree programme and create degree programmes with base classification rules. The 'add officer' form provides a dynamic list of checkboxes linked to all the degree programmes that do not yet have an officer assigned to them. This both allows the officer to be assigned to multiple programmes and ensures that the officer is not assigned to a programme that already has a classification officer, ensuring accuracy as required in D.1

and scalability as required in D.2. Additionally, the post route for this form uses bcrypt to hash the password before saving it to the database, ensuring that base security measures are met (D.3).

The 'add programme' form allows the administrator to define classification rules for a programme. I ensured that the classification weighting rules, boundary rules etc. were not hard-coded to meet the scalability requirement (D.2), allowing programmes with differing classification rules to be incorporated, satisfying requirement C.9. I did, however, autofill the input fields with the base classification rules to improve the user experience and save them time when inputting standard programmes. In contrast to the 'add officer' form, this one displays a checkbox list of all officers on the system to satisfy the requirement that one officer can be assigned to many programmes (B.4). And finally, the post route performs a check to ensure that the sum of the yearly weightings equal exactly 1.0 before saving, ensuring accuracy as required by requirement D.1.

The buttons to view all degree programmes and view all officers use SQL statements to fetch lists of data, which are looped through in the EJS to be displayed on screen. I also included a search bar and a filter drop-down for user experience. The user can quickly locate an officer or programme regardless of how large the dataset grows, which makes the app highly scalable (D.2). The filter dropdown allows the user to filter by assignment status, allowing the administrator to efficiently prioritise and assign tasks. It was a challenge to get the search and filter forms to work together, but after research, I added an onchange="this.form.submit()" to rectify this problem and streamline the user experience.

When the administrator selects a programme or officer from the list they are taken to a personalised page where they can view or edit the data, satisfying the management requirement B.2. Once again, the input fields are pre-filled with the current data in the database for user experience (D.4). However, I ensured that the password field was not auto filled to preserve base security measures (D.3). Therefore, the post route first checks if the password input field is empty. If so, it is not included in the req.body and not updated in the database, retaining the old password value. If it is not empty, the new password is hashed and updated in the database.

When the administrator selects a programme, they can both edit its base info that is stored in the programmes table and create or edit modules for that programme. I chose to do this in two separate forms to make the post route clearer and more efficient. Known values are prefilled, and dropdowns are used where possible to ensure accuracy and minimise human input errors (D.1). The form to edit modules was a challenge to implement because I wanted the modules to appear in a table format, detailing the module code, name, level and credits but I also didn't want to unintentionally cap the number of modules that could be added, considering the numerous optional modules that are associated with some degree programmes. Considering that the lowest credits

assigned to a module is 20, I created a table with a base of 18 rows to allow for the typical 360 credits associated with undergraduate degrees. After some research, I decided that the best way to do this was to enable the user to dynamically add or remove rows in the table using client-side JavaScript in the EJS template. The logic for this form was also challenging because of how the form data is returned. I chose a parallel array approach because of its simplicity and because there are no risk factors in the EJS template that could affect its efficacy, such as conditionally hidden fields. If the EJS changed, a dynamic key approach may need to be considered for accuracy. Each column was returned as a separate array, so they had to be looped through and matched by index before being inserted or updated in the database. The business logic also had to check for empty rows using the .filter() method to prevent the storage of incomplete data and meet the accuracy requirement (D.1).

IV. *Primary Actor*

The Classification Officer's dashboard contains a list of only the programmes that the specific officer is responsible for and some statistical summary data for each cohort, satisfying requirement C.1. I also included a search bar to account for future scalability to help the officer navigate large numbers of assignments efficiently. Programmes are assigned based on graduation cohort year, so officers can search by academic year to filter their current year assignments as well as by programme title and award type. The programme-specific statistical data displays the total number of students, the number of approved students and the number awaiting approval, to give the officer a scannable view of what needs to be completed, allowing them to better handle deadlines.

When the officer clicks on a degree programme, they are brought to a list of students in that cohort, displaying only identifiable data for each student, including their name, student number and the status of their classification review. This page also allows the officer to search by student name or number and filter the student list based on review status. This allows the officer to prioritise tasks by filtering to see only the student records that are flagged for manual review due to mitigating circumstances or a borderline automated classification result, as well as the records that are awaiting approval or have already been approved.

A chart was also included on the student list page to satisfy requirement C.6. It was implemented using chart.js to give a visual representation of classification distribution across the programme cohort. A significant challenge was that the calculateClassification function is not executed until the officer views an individual student's record, and only for one student at a time, which caused my graph logic in the view student list controller to return every proposed classification as the default value "Pending". To fix this, I had to query the database using the JSON_ARRAYAGG command to get a list of all students and their grades as a nested object. The grades object was extracted and passed into the calculateClassification function, as well as the list of

classification rules. The resulting classifications were collected into an array, which was then looped through and tallied within the web controller to produce the data for populating the chart, ensuring a deterministic overview of the cohort's performance as per requirement D.1.

The student list page also satisfies the Create and Delete parts of the cohort CRUD requirement C.2. It contains an add student button, which takes the officer to a form with input fields for a student's basic identity information. The delete requirement is met in the form of an archive button. Instead of completely deleting the student's record from the database, the archive button soft deletes the student by setting the is_deleted column to true in the database. The logic then uses this to remove the student from the user interface while they are still present in the database. I chose to implement the delete functionality in this way for students, programmes and officers to maintain data integrity across the app and to prevent the creation of orphaned data. In a real-world scenario, historical data may need to be accessed long after the entity has been 'deleted', so the safest and most practical option for data consistency and accuracy was to archive the entities instead of hard deleting them.

The Read and Update CRUD specifications as stipulated in requirement C.2 are met when the officer clicks the button to view an individual student's record. This page displays a detailed account of the student's record, including personal details, academic record and classification information. I chose to split these sections into three separate forms so that the officer can edit each dataset separately and to simplify the processing logic, improving the user experience. This was accomplished by wrapping each form in a fieldset tag, which disables editing of the form until the 'edit button' is clicked, triggering the execution of a client-side JavaScript function.

In the form to edit the student's personal details, I chose to make the student's degree programme and cohort year uneditable. Ultimately, a classification officer can only access the student records of students on programmes that they are responsible for. Given that programmes are assigned to officers by cohort graduation year, allowing the officer to edit these fields would jeopardise data integrity and accuracy. The responsibilities of a classification officer do not have the scope to transfer students outside of their assigned programmes. Additionally, if an officer transferred a student mid-way through their degree and their new programme had different classification rules, it would not be in line with the requirement to ensure deterministic and consistent rule execution. Therefore, it makes more sense for this task to be handled by a third actor in future iterations of the web app.

Similarly, the form for managing a student's modules and grades only has a select number of editable fields. The definitions of module codes, names and credits are the responsibility of the institutional administrator, so to preserve data integrity, these cannot be edited by the officer. Instead, the form displays a list of the modules associated with

the specified degree programme as defined by the administrator. There is a checkbox beside each module that can be checked if the student was registered on that module or unchecked if they weren't. The officer can also edit the student's grades and declare their resit status. I had also considered making grades outside of the student's current level uneditable. However, to allow for the possibility of grade challenges, I decided not to implement this feature. This feature could be reconsidered depending on the individual university's grade challenge deadline policies.

The form to edit module marks was also challenging due to the table format. In order to prefill the form with known student grades, I had to match up the module with the entry in the grades table and set the checked value depending on whether a grade entry existed for that programme and student_id. The EJS and business logic use dynamic field names instead of returning parallel arrays because it is more accurate in cases where the user may only select some modules using the checkbox. When the form is submitted, an array of selected modules is constructed by considering the checkbox data. All modules for that programme are then looped through, retrieving each module ID and its checkbox status. If the module is checked, the mark and resit fields are defined. The data is then inserted or updated using the UPSERT approach to avoid duplicate entries and ensure data consistency and accuracy as requested in requirement D.1.

The Classification details form displays the student's review status, their overall percentage, their classification, a notification of why the student's automated classification was flagged for manual review if applicable, an indication of its manual override status and decision rationale, satisfying requirement C.5. The form has a button which makes the form editable, allowing the classification officer to override the system's automated proposed classification, as the specification requires in C.3 and C.4. Upon editing of this data, the manual override field is automatically set to true. I chose to make the student's overall average an uneditable field because this would pose an accuracy risk (D.1). However, the officer can override the classification and explain the rationale, which allows for better traceability of decisions as suggested in requirement D.4. There is also an approve button which changes the review status to 'approved', updating the final_classification database field and the officer's dashboard statistics.

As requirement C.3 expects, a proposed classification is generated automatically by the system. The business logic for this lives in the utils folder and dynamically calculates the classification based on the student's grades and the programme-specific classification rules, satisfying requirements C.8, C.9 and D.1. The programmes table in my database contains the fields for classification calculation, including yearly weights, boundaries, fail allowances, dissertation pass mark and resit capping, allowing the rules to be defined per programme (C.9). The calculateClassification function references these values by their field name rather than hard coding them allowing for dynamic classification calculations. An array of the student's grades and an array of their programme's

classification rules are captured using SQL statements and passed into the function. I then looped through them and separated them by level so that the correct weightings could be applied and yearly averages calculated. Before calculating the yearly averages, I made sure to account for grades that were affected by resit capping where applicable. I also took into account dissertation grades where the mark did not meet the minimum pass mark and calculated the total credits taken over the three years to ensure that the student met the criteria for honours consideration. I then checked to see if any failed modules were eligible for compensation based on the programme-specific rule stored in the database.

The level averages were then used to calculate the overall average to automate the classification. Before entering the main classification assignment conditional statement, I first checked that each honours condition was met, including whether the student had passed the dissertation, passed all 3 years and that they had completed 360 credits, as is typical for undergraduate degrees. If these conditions are not met, the classification assignment if block is skipped. Otherwise, the if block assigns classifications dynamically using the programme-specific boundary values stored in the database. This block of code also determines whether the student's average is considered borderline, so it can be flagged for manual review. If a student has not completed the necessary credits and cannot yet be considered for classification, their classification is set to pending and displayed as such on the user interface. The calculateClassification function then returns the necessary results to be used in the student record page and displayed through the EJS template.

The automatic calculation of a student's proposed classification was not hard-coded to ensure deterministic and consistent rule execution as outlined in requirement D.1 and allow for future scalability as outlined in requirement D.2. The system will dynamically handle the addition of future degree programmes with varying classification rules without requiring human intervention, allowing for more efficient and accurate management of cohorts. In addition to the classification engine, the execution of client-side and server-side elements resulted in an app with a modern and responsive user experience as stipulated in requirement D.4. To summarise, this app satisfies the full list of requirements outlined in the specification, except C.7 due to time constraints. The addition of export options would add convenience to the user experience, but without this feature, the app is still completely functional and responsive for future growth and adaptations.